



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

REPORTE
Segundo Proyecto: OjalvoMusic

PRESENTA

Isidro Armando Abelló García
426009038

PROFESOR

Dr. Canek Peláez Valdés

ASIGNATURA

Modelado y Programación

13 de noviembre de 2026

1. Introducción

El proyecto consiste en una biblioteca musical de escritorio, `ojalvomusic`. El enunciado pide cuatro componentes:

1. Una **base de datos** SQLite con un esquema fijo (`rolas.sql`), que no se puede modificar.
2. Un **minero** que recorre un directorio, lee las etiquetas ID3v2.4 de los archivos MP3 y guarda su información en la base de datos.
3. Una **interfaz gráfica** para consultar y modificar esos datos: editar rolas y álbumes, definir intérpretes como personas o grupos y agregar personas a grupos.
4. Una forma de **buscar** rolas por campos sin escribir SQL.

Como punto extra se pide **reproducir** los MP3. Una regla atraviesa todo el proyecto: los archivos MP3 nunca se modifican; cualquier cambio ocurre solo en la base de datos.

Desde el principio decidí escribirlo en **C#**, porque su orientación a objetos lo hace ideal para un proyecto con tantas piezas que colaboran entre sí. Quedaba elegir el entorno gráfico; después de investigar las opciones disponibles escogí **Avalonia**, por ser multiplataforma.

Este reporte describe cómo se desarrolló el proyecto: la metodología que seguí, los siete problemas en los que dividí el trabajo, las técnicas y patrones de diseño que usé en cada uno, y lo que aprendí. El uso del programa está en el README y en el manual de usuario (archivos `README.md` y `MANUAL.md` del repositorio).

2. Desarrollo

2.1. Metodología

A partir de la experiencia de mi proyecto anterior, y después de analizar el problema, decidí usar una metodología **ágil**, para ser más versátil al abordar cada dificultad. Separé el problema principal en siete subproblemas "grandes" (tabla 1) y a cada uno le apliqué un pequeño esquema de **cascada**: análisis, diseño, desarrollo y pruebas.

#	**Problema**	**Fechas (2026)**
0	Control y acceso a la base de datos	16 de octubre
1	Minero	18 al 23 de octubre
2	Controlador de entrada	27 al 31 de octubre
—	Reestructuración de la solución	1 de noviembre
3	Interfaz gráfica en Avalonia	3 al 7 de noviembre
4	Buscador de rolas	8 al 10 de noviembre
5	Personas y grupos	10 y 11 de noviembre
6	Reproductor de música (punto extra)	11 y 12 de noviembre

Tabla 1: Problemas en los que se dividió el trabajo.

En cada paso traté de aplicar **"code a little, test a little"**: escribir un trozo pequeño de código y probarlo antes de seguir. El historial lo refleja: son alrededor de 60 commits pequeños, cada uno con un solo propósito, y el proyecto termina con 311 pruebas.

2.2. Problema 0: control y acceso a la base de datos

Análisis. Había que crear el acceso a la base de datos y permitir su modificación. El esquema ya estaba dado y no se podía cambiar, así que el trabajo era envolverlo bien. También había que decidir qué hace única a cada cosa: una rola es su ruta, un álbum es su nombre más su directorio, y un intérprete es su nombre exacto.

Diseño. El profesor Canek sugirió usar el patrón **DAO** (*Data Access Object*) y decidí hacerle caso. Cada tabla tiene un DAO que encapsula su SQL detrás de una interfaz (`IRolaDao`, `IAlbumDao`, `IPerformerDao`), de modo que el resto del programa no escribe consultas. Agregué además un manejador de transacciones, `UnitOfWork`, para garantizar limpieza y seguridad al tocar la base:

- Es dueño de una sola conexión y de la transacción en curso. Los DAOs reciben el `UnitOfWork`, no la conexión.
- Implementa `IDisposable`: al salir del bloque `using`, una transacción sin confirmar se deshace sola.
- Cada operación abre su propio `UnitOfWork`, así que una conexión nunca se comparte entre hilos.

Desarrollo. Comencé por los modelos y los DAOs de rolas, álbumes e intérpretes, que es lo que llenaría el minero en el problema 1. La base se crea a partir de `rolas.sql`, que viaja intacto como recurso embebido (`DatabaseInitializer`). Todas las consultas usan parámetros; nunca se concatena texto, lo que brinda seguridad contra ataques de inyección de SQL (*SQL injection*).

Pruebas. Decidí delegar las pruebas de esta parte hasta que el minero estuviera listo, porque el minero es su primer usuario real. Con él llegaron las pruebas del inicializador (creación, segunda llamada, esquemas ajenos, bases corruptas, llaves foráneas) y más adelante las de cada DAO.

Técnicas y patrones: DAO, *Unit of Work*, patrón *disposable*, programación contra interfaces, consultas parametrizadas, transacciones.

2.3. Problema 1: minero

Análisis. El minero debe encontrar todos los MP3 de un directorio, de forma recursiva; leer sus etiquetas sin modificarlos; inferir las rutas de rolas y álbumes; y no duplicar lo que ya está en la base. Además no puede congelar la interfaz mientras trabaja, y debe poder cancelarse.

Diseño. Subdividí el problema en varias clases, cada una con una sola responsabilidad:

- `FileScanner` obtiene los archivos.
- `TagReader` lee las etiquetas (con la biblioteca TagLibSharp, sobre una abstracción de archivo de solo lectura).
- `Miner` orquesta la minería e integra lo resuelto en el problema 0.

El minero trabaja en dos fases: primero lee las etiquetas y después guarda todo en una transacción, de modo que si algo falla a la mitad, la base no queda a medias. Una decisión importante fue comprobar si una rola ya existe **consultando la base de datos**, en lugar de copiar las rutas a memoria. Esto prioriza la memoria y la simplicidad del código sobre la velocidad: se cambia una búsqueda $O(1)$ en una tabla hash por una $O(\log n)$ en el índice de la base.

Garanticé además que la minería fuera segura con hilos. Corre fuera del hilo de la interfaz (`Task.Run`), un semáforo (`SemaphoreSlim`) impide dos minerías simultáneas, el avance se reporta con `IProgress` y la cancelación es cooperativa, con `CancellationToken`.

Desarrollo. Primero hice una "maqueta" del minero, sin probar, junto con el inicializador de la base y los contratos (interfaces y objetos de transferencia como `TagInfo` y `MiningResult`). En ese paso corregí también el error de la primera versión, que sacaba prácticamente toda la base para hacer comprobaciones en C#.

Pruebas. Fue el bloque de pruebas más grande del proyecto. El minero recibe sus piezas por **inyección de dependencias**, y eso permitió usar técnicas de *mocking*: en las pruebas se le entregan dobles (`FakeFileScanner`, `FakeTagReader`) en lugar de las piezas reales. Se probaron el minado normal, la segunda corrida, cancelaciones, archivos corruptos, duplicados y el minado simultáneo. Una prueba de integración junta las tres piezas reales sobre un conjunto de MP3 generados (`Dataset/`), y otra comprueba que los MP3 no se modifican.

Técnicas y patrones: responsabilidad única, inyección de dependencias por constructor, dobles de prueba, DTOs, tareas asíncronas, semáforos, cancelación cooperativa.

2.4. Problema 2: controlador de entrada

Análisis. El programa se invoca como `ojalvomusic [opciones] [ruta]` y, según los parámetros, debe minar, abrir la interfaz, hacer ambas cosas o mostrar la ayuda. Los errores de uso deben reportarse de forma limpia y con un código de salida.

Diseño. Para seguir el patrón **MVC** creé un `Controller` que orquesta lo que hace el programa según la entrada. El análisis de los parámetros quedó aparte, en `ArgumentParser`: una función pura que recibe los argumentos y regresa las opciones, o lanza una `UsageException`. Implementé también una **bitácora** para tener mejor control del flujo del programa. Es una interfaz (`ILog`) con dos implementaciones: `ConsoleLog`, que se activa con `-l`, y `NullLog`, que aplica el patrón *Null Object* para no llenar el código de comprobaciones.

El controlador también usa inyección de dependencias: recibe los flujos de salida, una función que fabrica el minero y una acción que abre la vista. Así no conoce la interfaz gráfica y puede probarse sin ella.

Desarrollo. Se hizo en este orden: punto de entrada, parser de argumentos, controlador y bitácora. Los códigos de salida siguen las convenciones: 0 éxito, 1 error, 2 uso inválido y 130 cancelado con Ctrl+C.

Pruebas. Cada pieza tiene las suyas. Las del parser cubren cada opción y cada error de uso. Las del controlador le inyectan un `StringWriter` y un minero armado con piezas falsas, y revisan mensajes y códigos de salida.

Técnicas y patrones: MVC, *Null Object*, inyección de dependencias con delegados (`Func`, `Action`), funciones puras, excepciones propias.

2.5. Reestructuración de la solución

Mi idea original era tener un solo proyecto. Cuando estaba integrando Avalonia me di cuenta de que eso sería problemático: sus paquetes podían "contaminar" el proyecto, de modo que para correr pruebas o ejecutar solo cosas del modelo se iba a cargar la interfaz. Además todo quedaría desorganizado, e incluso podría violarse la separación de responsabilidades.

Opté entonces por una solución con **tres proyectos**:

```
MusicLib/         el modelo, como biblioteca
MusicLib.UI/      el punto de entrada y la interfaz
MusicLib.Tests/   las pruebas
```

Las dependencias van en un solo sentido: la interfaz y las pruebas conocen al modelo, pero el modelo no conoce a ninguna de las dos.

2.6. Problema 3: interfaz gráfica en Avalonia

Análisis. Aquí el problema era, sobre todo, aprender: los básicos de Avalonia, su lenguaje de marcado (XAML) y la forma en que una ventana se comunica con el código.

Diseño. Decidí aplicar, con perspectivas a futuro, el patrón **Modelo-Vista-VistaModelo (MVVM)** en lugar de MVC, porque Avalonia cuenta con una herramienta (Community Toolkit) que hace más fácil y natural usarlo. En este patrón, el VistaModelo se encarga más activamente de controlar el estado de su ventana que de comunicarse con el modelo, pero cumple los mismos objetivos de separación de responsabilidades y organización que MVC. Como me dijo el profesor Canek: "(...) MVVM es MVC con pasos extra (...)".

La vista se enlaza al VistaModelo con *bindings*: cuando una propiedad cambia, la ventana se actualiza sola, y los botones ejecutan comandos del VistaModelo. Para lo que el VistaModelo no debe hacer directamente, definí servicios que se le inyectan: uno para elegir carpetas y una clase auxiliar para abrir ventanas modales, pensada para los problemas siguientes.

Desarrollo. Primero hice maquetas, solo visuales, de las ventanas que iba a usar: la principal, la búsqueda avanzada, los formularios y el editor. Después conecté la primera funcionalidad real, el ciclo de vida del botón Minar: elegir la carpeta, mostrar el avance, cancelar y cargar las rolas en la tabla.

Pruebas. La interfaz se verificó a mano, ventana por ventana. El modelo siguió cubierto por sus pruebas automáticas, que no cargan Avalonia gracias a la reestructuración.

Técnicas y patrones: MVVM, *Observer* (notificación de cambios y eventos), *Command*, enlace de datos, servicios inyectados.

2.7. Problema 4: buscador de rolas

Análisis. El problema se dividió en dos: la búsqueda general, con un solo texto que se busca en título, intérprete, álbum y género; y la búsqueda avanzada, por campos separados y rango de años. El usuario nunca escribe SQL.

Diseño. La infraestructura ya estaba hecha, así que solo hubo que extender los DAOs para que hicieran consultas no triviales, con `JOIN` entre rolas, intérpretes y álbumes. Los criterios viajan en un objeto (`RolaSearch`), y una clase pequeña (`RolaSearchSql`) arma la consulta agregando una condición fija por cada criterio con valor, siempre con parámetros. Para que la búsqueda no distinga mayúsculas ni acentos, un normalizador de texto se registra como función dentro de SQLite, sin tocar el esquema.

Desarrollo. Primero el DAO, después la búsqueda general conectada a la vista y por último la ventana de búsqueda avanzada con su VistaModelo. En la vista bastó acomodar los *bindings*. La granularidad y la separación entre Modelo, Vista y VistaModelo resultaron muy convenientes: cada cambio tocó pocas piezas.

Con la tabla ya funcionando agregué sobre la misma base la **edición de rolas y álbumes**: el `UPDATE` en los DAOs, una clase del modelo (`LibraryEditor`) que valida y guarda cada cambio en una transacción, los editores en la interfaz y la portada de la rola seleccionada.

Pruebas. Se probaron el normalizador, el armado del SQL y el DAO; el resto de la búsqueda se verificó en la vista. El editor tiene pruebas propias para cada regla.

Técnicas y patrones: objeto de criterios, consultas parametrizadas armadas dinámicamente, función SQL definida por el programa, capa de servicio con una transacción por operación, *bindings*.

2.8. Problema 5: personas y grupos

Análisis. El manejo de este problema se parece mucho al anterior. Lo separé en dos: el de las personas y el de los grupos. Para cada uno elaboré una serie de reglas, que están especificadas en el manual. Por ejemplo: qué pasa al agregar una persona cuyo nombre ya es un intérprete, qué se borra al eliminar un grupo, y cómo se validan las fechas.

Diseño. Extendí los DAOs (`PersonDao`, `GroupDao` y `MembershipDao` para la tabla `in_group`) y agregué dos editores en el modelo, `PersonsEditor` y `GroupsEditor`. Cada operación valida y cambia la base en una sola transacción. Las reglas de validación viven en el modelo y los VistaModelos las reutilizan: el botón Guardar se deshabilita mientras los datos sean inválidos, y la ventana dice el motivo.

Desarrollo. Escribí un plan de 17 pasos, un commit por paso: primero el modelo y después la interfaz. Esta última incluye las ventanas de lista, un mismo formulario para agregar y editar, un diálogo genérico de confirmación antes de borrar y una lista doble para elegir a los miembros de un grupo.

Pruebas. Cada DAO y cada editor llegaron con sus pruebas, que cubren conflictos de nombres, fechas inválidas, borrados y membresías. La interfaz se verificó a mano.

Técnicas y patrones: DAO, capa de servicio, transacciones, validación compartida entre modelo y VistaModelo, comandos con condición de ejecución, reutilización de ventanas.

2.9. Problema 6: reproductor de música

Análisis. Avalonia no reproduce audio, así que el punto extra se redujo prácticamente al uso de una biblioteca. Aun así hubo problemas no triviales, relacionados con los hilos de ejecución, al implementar la barra de progreso y lograr que sonara adecuadamente.

Diseño. Elegí LibVLCSharp, que usa el VLC instalado en el sistema. El reproductor queda detrás de una interfaz (`IAudioPlayer`), con una implementación que adapta la biblioteca (`VlcAudioPlayer`) y un VistaModelo propio. Si VLC no está instalado, el programa no se cae: el reproductor queda deshabilitado y lo avisa.

El problema de los hilos es que VLC avisa desde un hilo suyo, y la interfaz solo puede tocarse desde el suyo. Lo resolví de dos formas:

- El aviso de "terminó la rola" se pasa al hilo de la interfaz antes de atenderlo.
- La barra de progreso no escucha a VLC; un temporizador del hilo de la interfaz le pregunta la posición varias veces por segundo.

Desarrollo. En dos pasos: primero reproducir y pausar la rola seleccionada, y después la barra de progreso con el tiempo y el salto a cualquier punto.

Pruebas. No hay pruebas automáticas de audio; se verificó a mano: reproducir, pausar, reanudar, cambiar de rola, llegar al final, saltar con la barra y abrir el programa sin VLC.

Técnicas y patrones: *Adapter*, inyección de dependencias, eventos, paso de mensajes al hilo de la interfaz, temporizador de sondeo.

3. Diagramas UML simplificados del proyecto

Esta sección resume el diseño en tres diagramas de clases, uno por cada capa del programa. Están simplificados a propósito: cada clase muestra solo sus miembros principales, sin tipos ni parámetros, y se omiten las clases auxiliares.

La figura 1 muestra los modelos y el acceso a la base de datos. El `UnitOfWork` es dueño de la conexión y de los seis DAOs, y los tres editores lo usan para cambiar la base en una sola transacción.

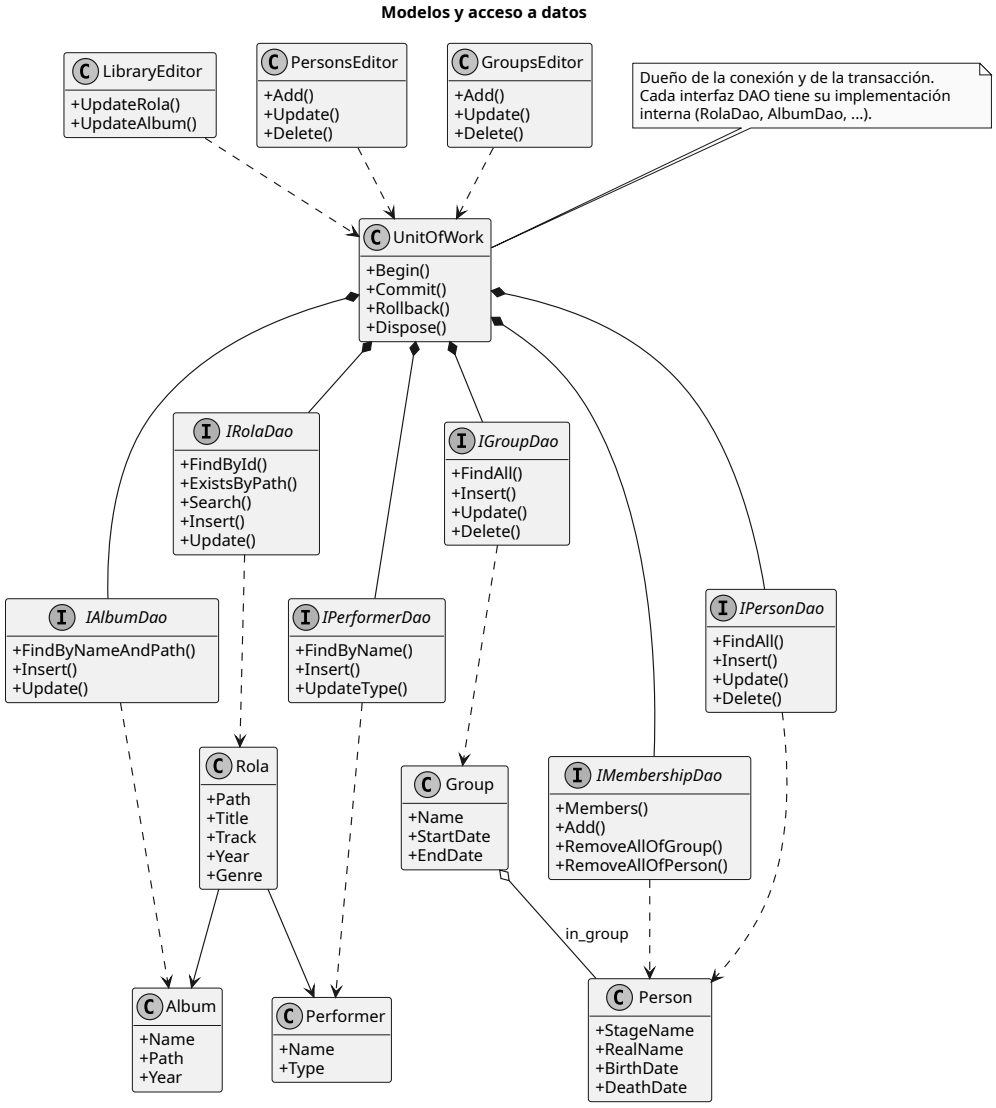


Figura 1: Modelos y acceso a datos (patrones DAO y *Unit of Work*).

La figura 2 muestra el camino desde el punto de entrada: el `Controller` depura los parámetros, crea el minero y le entrega la bitácora. El minero solo conoce las interfaces de sus piezas, lo que permite sustituirlas en las pruebas.

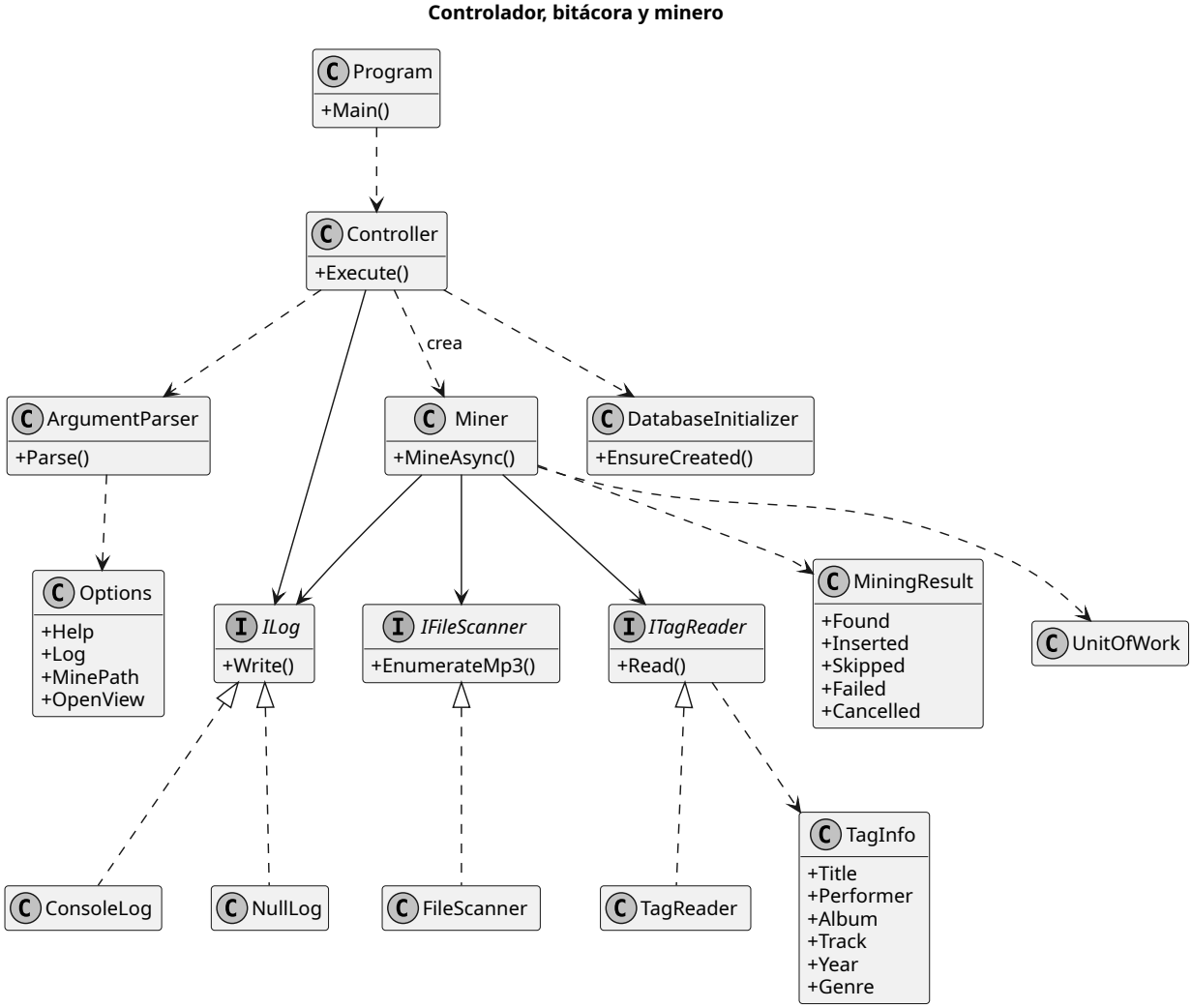


Figura 2: Controlador, bitácora y minero.

La figura 3 muestra la interfaz gráfica. El VistaModelo principal recibe sus servicios por inyección, y el servicio de diálogos crea los VistaModelos de las ventanas secundarias, que a su vez usan los editores del modelo. Las ventanas no se dibujan una por una: cada VistaModelo tiene la suya.

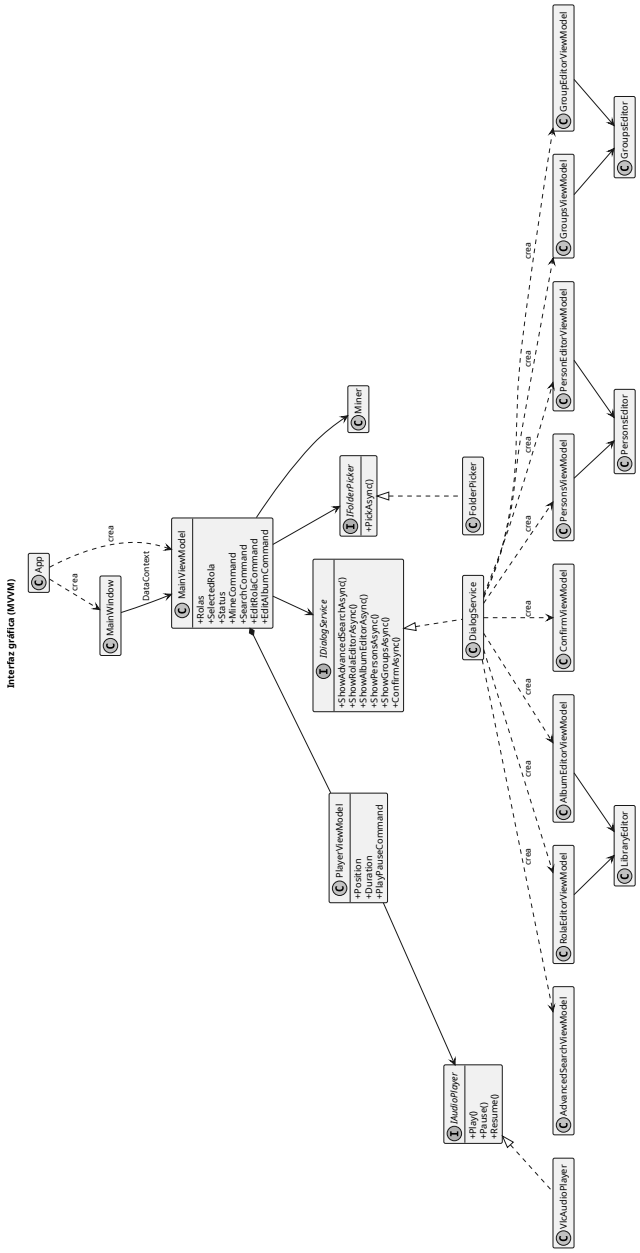


Figura 3: Interfaz gráfica (patrón MVVM). El diagrama está girado para aprovechar la página.

4. Conclusiones

El proyecto fue muy importante para mi formación. Me obligó a pensar en el diseño antes que en el código, y cada problema posterior resultó más fácil gracias a las decisiones tomadas en los anteriores: para la búsqueda y para las personas y grupos, la infraestructura ya estaba hecha, por ejemplo.

Aprendí patrones y técnicas nuevas:

- **Bases de datos:** el patrón DAO, el lenguaje SQL (en particular con SQLite) y el manejo de transacciones.
- **C#:** interfaces y bibliotecas útiles del lenguaje, delegados y eventos, el manejo de tipos anulables, el patrón *disposable* y sus herramientas de pruebas.
- **Concurrencia:** hilos, tareas (`Task`), promesas y semáforos.
- **Pruebas:** inyección de dependencias y *mocking*.
- **Interfaz gráfica:** Avalonia, el patrón MVVM y Community Toolkit.

Sobre la metodología, dividir el trabajo en problemas "grandes" y aplicar a cada uno una pequeña cascada funcionó bien: siempre supe qué estaba resolviendo y pude cambiar de rumbo cuando hizo falta, como en la reestructuración de la solución.

Me hubiera gustado implementar un mejor buscador, en el que las personas y los grupos se asociaran a las búsquedas de rolas. También me hubiera gustado hacer un diseño más creativo de la interfaz, para darle un toque de originalidad. Aun así, estoy muy conforme con el diseño general del programa.

Bibliografía

[1] Wikipedia. *Data access object*. `https://en.wikipedia.org/wiki/Data_access_object`

[2] NetMentor. *Unit of work pattern C#*. `https://www.netmentor.es/entrada/en/unit-of-work`

[3] SQLite. *Sitio oficial y documentación*. `https://www.sqlite.org/`

[4] Microsoft. *Microsoft.Data.Sqlite overview*. Microsoft Learn. `https://learn.microsoft.com/dotnet/standard/data/sqlite`

[5] Mono Project. *TagLib#* (taglib-sharp). GitHub. `https://github.com/mono/taglib-sharp`

[6] Microsoft. *Asynchronous programming with async and await*. Microsoft Learn. `https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/`

[7] xUnit.net. *Sitio oficial y documentación*. `https://xunit.net/`

[8] Microsoft. *Mocking for tests in C#*. Code-With-Engineering Playbook. `https://microsoft.github.io/code-with-engineering-playbook/automated-testing/unit-testing/mocking/`

[9] Refactoring.Guru. *Introduce Null Object*. `https://refactoring.guru/introduce-null-object`

[10] Wikipedia. *Model–view–controller*. `https://en.wikipedia.org/wiki/Model-view-controller`

[11] Avalonia. *Avalonia Docs*. `https://docs.avaloniaui.net/`

[12] Wikipedia. *Model–view–viewmodel*. `https://en.wikipedia.org/wiki/Model-view-viewmodel`

[13] Avalonia. *The MVVM pattern*. Avalonia Docs. `https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern`

[14] Microsoft. *Introduction to the MVVM Toolkit*. Microsoft Learn. `https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/`

[15] VideoLAN. *LibVLCSharp*. GitHub. https://github.com/videolan/libvlcsharp

[16] Jovanović, M. *Milan Jovanović* (canal de YouTube sobre arquitectura y patrones en .NET). https://www.youtube.com/@MilanJovanovicTech

Todas las fuentes se consultaron entre octubre y noviembre de 2026.